

Tarefa 3: Implementação e Análise de Convergência para o Cálculo de π

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Maio de 2026

1. Introdução

Este relatório descreve a implementação de um programa em linguagem C capaz de calcular uma aproximação do número π por meio da Série de Leibniz. O objetivo central é observar como a acurácia da aproximação evolui em função do número de iterações realizadas e como o tempo de processamento acompanha esse crescimento de forma proporcional.

A análise desse comportamento fornece uma base concreta para compreender um problema fundamental da computação científica: quanto mais precisão se exige de um cálculo iterativo, maior é o custo computacional associado. Tal dilema é central em aplicações de alto desempenho, como simulações físicas, modelos climáticos e treinamento de redes neurais profundas.

2. Fundamentação Teórica

2.1 Série de Leibniz

A Série de Leibniz é uma das representações mais simples de π em forma de soma infinita, enunciada pela seguinte expressão:

$$\pi/4 = 1 - 1/3 + 1/5 - 1/7 + 1/9 - \dots$$

Cada termo da série segue o padrão $(-1)^n / (2n + 1)$, com $n \geq 0$. A série converge para $\pi/4$, de modo que o valor aproximado de π é obtido multiplicando-se a soma parcial por 4. Apesar de matematicamente elegante, a série apresenta convergência lenta: para cada ordem de grandeza adicional de iterações (fator $\times 10$), obtém-se apenas uma casa decimal a mais de precisão.

2.2 Convergência e Complexidade

A taxa de convergência da Série de Leibniz é $O(1/N)$, onde N é o número de iterações. Isso significa que o erro absoluto em relação a π decresce linearmente com o inverso de N . Em termos práticos, multiplicar N por 10 divide o erro por 10 — ganho de uma única casa decimal. Esse padrão torna o custo computacional progressivamente mais caro conforme se exige maior precisão.

3. Implementação

3.1 Estrutura do Código

O programa foi escrito em C e compila sem dependências externas além da biblioteca matemática padrão. A medição de tempo é realizada com `clock_gettime(CLOCK_MONOTONIC)`, garantindo alta resolução e independência de ajustes de relógio do sistema. A função central `calcular_pi()` recebe o número de iterações N e retorna a aproximação calculada:

```
#define _POSIX_C_SOURCE 199309L
#include <stdio.h>
#include <math.h>
#include <time.h>

#define PI_REF 3.14159265358979323846
#define BILLION 1000000000.0

static double calcular_pi(long long n) {
    double soma = 0.0;
    for (long long i = 0; i < n; i++) {
        double termo = 1.0 / (2 * i + 1);
        if (i % 2 == 0)
            soma += termo;
        else
            soma -= termo;
    }
    return 4.0 * soma;
}
```

O programa testa oito valores de N — de 100 a 1.000.000.000 — registrando, para cada caso, o valor aproximado de π , o erro absoluto em relação ao valor de referência (3,14159265358979...) e o tempo de execução em segundos.

3.2 Compilação e Execução

O projeto inclui um Makefile que automatiza compilação e execução:

```
make all      # compila o projeto
make run      # executa o binário ./pi
```

4. Resultados Obtidos

A tabela a seguir apresenta os resultados coletados durante a execução do programa em ambiente sequencial de thread única:

Tabela 1 — Resultados da aproximação de π pela Série de Leibniz

Iterações	π Aproximado	Erro Absoluto	Tempo (s)
100	3,13159290355855	$1,00 \times 10^{-2}$	0,000000
1.000	3,14059265383979	$1,00 \times 10^{-3}$	0,000001

Iterações	π Aproximado	Erro Absoluto	Tempo (s)
10.000	3,14149265359003	$1,00 \times 10^{-4}$	0,000013
100.000	3,14158265358972	$1,00 \times 10^{-5}$	0,000128
1.000.000	3,14159165358977	$1,00 \times 10^{-6}$	0,001096
10.000.000	3,14159255358979	$1,00 \times 10^{-7}$	0,010606
100.000.000	3,14159264358933	$1,00 \times 10^{-8}$	0,102906
1.000.000.000	3,14159265258805	$1,00 \times 10^{-9}$	1,029611

Os dados confirmam a convergência linear prevista teoricamente: o erro absoluto decresce por um fator de 10 a cada vez que N é multiplicado por 10. O tempo de execução acompanha o mesmo fator de escala, crescendo de forma estritamente proporcional ao número de iterações.

5. Análise

5.1 Padrão de Convergência

A relação entre o fator de aumento de N e a melhoria de precisão pode ser resumida conforme a tabela abaixo:

Tabela 2 — Relação entre fator de N e ganho de precisão

Fator de N	Melhoria no Erro	Ganho de Precisão
$\times 10$	$\div 10$	Uma casa decimal
$\times 100$	$\div 100$	Duas casas decimais
$\times 1.000$	$\div 1.000$	Três casas decimais
$\times 1.000.000$	$\div 1.000.000$	Seis casas decimais

Esse padrão evidencia a natureza linear da convergência em escala logarítmica. Para alcançar 9 casas decimais de precisão — como obtido com 1 bilhão de iterações — o programa consome aproximadamente 1 segundo em modo sequencial. Estender a precisão para 12 casas decimais exigiria 1 trilhão de iterações, resultando em um tempo estimado de cerca de 17 minutos em uma única thread.

5.2 Custo Computacional e Motivação para Paralelismo

O experimento evidencia um padrão recorrente em computação de alto desempenho: mais precisão exige mais iterações, que por sua vez demandam mais tempo de processamento. Aplicações reais — como simulações de dinâmica de fluidos, previsão do tempo e treinamento de modelos de aprendizado de máquina

— precisam de respostas dentro de janelas de tempo viáveis, e não após horas de processamento.

A solução natural para esse gargalo é a programação paralela. O loop da Série de Leibniz pode ser particionado em P blocos independentes, cada um atribuído a uma thread distinta. Cada thread calcula sua fatia da soma parcial, e os resultados são combinados ao final por meio de uma operação de redução. Com P threads, o tempo de execução cai para aproximadamente $T_{\text{sequencial}} / P$, mantendo a mesma precisão final.

Assim, o cálculo que leva 1 segundo em um único núcleo poderia ser concluído em aproximadamente 0,125 segundo com 8 núcleos — ou em 0,031 segundo com 32 núcleos. Esse é o princípio central da programação paralela: não aumentar o que um núcleo faz, mas distribuir o trabalho entre múltiplos núcleos que operam simultaneamente.

6. Conclusão

A implementação da Série de Leibniz em C demonstrou, de forma concreta, a relação direta entre número de iterações, precisão e tempo de execução. A convergência lenta da série — uma casa decimal por ordem de grandeza de iterações — ilustra um desafio fundamental da computação científica: o custo de mais precisão cresce proporcionalmente ao ganho obtido.

Essa observação motiva diretamente o estudo e a aplicação de técnicas de programação paralela. Ao dividir o trabalho entre múltiplos núcleos de processamento, é possível manter a precisão desejada sem aumentar o tempo total de execução, tornando viáveis cálculos que seriam proibitivos em modo sequencial. O experimento serve, portanto, como ponto de partida para compreender por que sistemas modernos de alto desempenho fazem uso extensivo de paralelismo.

7. Código Fonte

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <math.h>
4  #include <time.h>
5
6  #define PI_REF 3.14159265358979323846
7  #define BILLION 1000000000.0
8
9  static double elapsed(struct timespec a, struct timespec b) {
10     return (b.tv_sec - a.tv_sec) + (b.tv_nsec - a.tv_nsec) / BILLION;
11 }
12
13 /* Série de Leibniz:  $\pi/4 = 1 - 1/3 + 1/5 - 1/7 + \dots$  */
14 static double calcular_pi(long long n) {
15     double soma = 0.0;
16     for (long long i = 0; i < n; i++) {
17         double termo = 1.0 / (2 * i + 1);
18         if (i % 2 == 0)
19             soma += termo;
20         else
21             soma -= termo;
22     }
23     return 4.0 * soma;
24 }
25
26 int main(void) {
27     long long iteracoes[] = {
28         100LL,
29         1000LL,
30         10000LL,
31         100000LL,
32         1000000LL,
33         10000000LL,
34         100000000LL,
35         1000000000LL
36     };
37     int n_casos = sizeof(iteracoes) / sizeof(iteracoes[0]);
38
39     printf("%-15s %-22s %-12s %-12s\n",
40         "Iterações", "π aproximado", "Erro absoluto", "Tempo (s)");
41     printf("%-15s %-22s %-12s %-12s\n",
42         "-----", "-----", "-----", "-----");
43
44     for (int k = 0; k < n_casos; k++) {
45         struct timespec ini, fim;
46         long long n = iteracoes[k];
47
48         clock_gettime(CLOCK_MONOTONIC, &ini);
49         double pi_aprox = calcular_pi(n);
50         clock_gettime(CLOCK_MONOTONIC, &fim);
51
52         double t = elapsed(ini, fim);
53         double erro = fabs(pi_aprox - PI_REF);
54
55         printf("%-15lld %.20f %-12.2e %.6f\n", n, pi_aprox, erro, t);
56     }
57
58     return 0;
59 }
60
```